University for the Creative Arts

BSBI School of Business and Innovation

Essay / Assignment Title: **Training Data Science Skills: From Data Cleaning to a Machine Learning Model**

Programme title: **MSc Data Analytics**

Name: **RAJEESH RAJAN**

Year: **2026**

CONTENTS

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname (Capital letters):

RAJEESH RAJAN

Date: 27/06/2026

Chapter One: Introduction

Data Quality Assessment, EDA, and Class Impurity Analysis

This section presents the data quality assessment, cleaning procedure, exploratory data analysis (EDA), and class impurity analysis for the provided classification dataset. The dataset contains 1,375 observations and five variables, including four numerical predictor variables: X1, X2, X3, and X4, and a binary target variable named Label, with class values of 0 and 1. The purpose of this analysis is to confirm that the dataset is appropriate for subsequent machine learning or statistical modelling, investigate the distributions and relationships among the predictor variables, identify important differences between the distributions of X1 and X4, and calculate class probabilities, Gini impurity, entropy, and the baseline misclassification rate.

Time Series Regression Analysis Using Polynomial Regression

This section investigates a time series regression dataset containing 1,000 observations and two numerical variables: *t*, representing time or an ordered continuous input variable, and *f(t)*, representing the target variable to be predicted. The aim of this task is to preprocess the dataset, explore its underlying trend, select an appropriate regression model, train and evaluate the selected model, and interpret the mathematical structure of the observed relationship. Although the data is ordered by time, it does not exhibit typical time series characteristics such as seasonality, repeated cycles, or dependence on previous target values. Instead, the exploratory analysis indicates a deterministic nonlinear relationship between *t* and *f(t)*. Therefore, Polynomial Regression was selected as the most suitable modelling method.

Chapter Two: Data Quality Assessment, EDA, and Class Impurity Analysis

2.1. Data Quality Checks and Cleaning Procedure

The first step was to inspect the structure of the dataset, including the number of rows, columns, data types, missing values, duplicate records, unique values, and descriptive statistics. All four feature variables (X1, X2, X3, and X4) were stored as numerical floating-point variables. The target variable Label was also initially stored as a floating-point variable because it contained one missing value.

The missing-value check showed that X1, X2, X3, and X4 did not contain any missing values. However, there was one missing value in the Label column. Since Label is the target variable, it could not be reliably imputed. Assigning either class 0 or class 1 would introduce an artificial label and could bias later classification results. Therefore, the row with the missing Label value was removed.

The duplicate check identified 25 exact duplicate records. Exact duplicate observations can lead to an over-representation of certain patterns in the data and may bias descriptive statistics or machine learning models. For this reason, all duplicate rows were removed.

The data were also checked for infinite values, such as positive infinity or negative infinity. No infinite values were found in any variable. In addition, the valid values of the target variable were checked. Apart from the one missing value, the Label column contained only the expected binary class values, 0 and 1. Therefore, no label recoding was necessary. After cleaning, the Label variable was converted from float to integer format.

A further inspection of the descriptive statistics revealed one extremely unusual observation with the following values:

$$X1 = -70,\ X2 = 120,\ X3 = -50,\ X4 = -80$$

```python
# 1. Remove rows with missing Label
clean_df = clean_df.dropna(subset=["Label"])
after_missing_removal = len(clean_df)

# 2. Remove exact duplicate rows
clean_df = clean_df.drop_duplicates()
after_duplicate_removal = len(clean_df)

# 3. Detect multivariate corrupted outlier using IQR
q1_clean = clean_df[feature_cols].quantile(0.25)
q3_clean = clean_df[feature_cols].quantile(0.75)
iqr_clean = q3_clean - q1_clean

lower_clean = q1_clean - 1.5 * iqr_clean
upper_clean = q3_clean + 1.5 * iqr_clean

outlier_flags_clean = (
    clean_df[feature_cols].lt(lower_clean) |
    clean_df[feature_cols].gt(upper_clean)
)

clean_df["outlier_feature_count"] = outlier_flags_clean.sum(axis=1)
```

```python
# Remove only rows extreme in at least three features
clean_df = clean_df[clean_df["outlier_feature_count"] < 3].copy()

# Remove helper column
clean_df = clean_df.drop(columns=["outlier_feature_count"])

# Convert Label from float to integer
clean_df["Label"] = clean_df["Label"].astype(int)

final_rows = len(clean_df)

print("Initial number of rows:", initial_rows)
print("Rows after missing Label removal:", after_missing_removal)
print("Rows after duplicate removal:", after_duplicate_removal)
print("Final number of clean rows:", final_rows)

print("\nMissing values after cleaning:")
print(clean_df.isnull().sum())

print("\nDuplicate rows after cleaning:")
print(clean_df.duplicated().sum())

print("\nClean Label distribution:")
print(clean_df["Label"].value_counts().sort_index())
```

Figure 1: Python code for data cleaning, outlier detection and removing using IQR method

This observation was highly extreme across all four variables simultaneously. It was substantially different from the remaining records and was likely caused by data entry, measurement, or data generation error. It was not treated as an ordinary univariate outlier because some features in the dataset naturally contain moderately extreme values. Instead, this record was removed because it was an extreme multivariate anomaly, meaning that all four feature values were implausibly distant from their normal ranges at the same time.

The cleaning procedure therefore involved three main steps: removing one row with a missing target label, removing 25 exact duplicate rows, and removing one clearly corrupted multivariate outlier. The final clean dataset contained 1,348 observations.

Cleaning step	Number of rows remaining
Original dataset	1,375
After removing missing Label value	1,374
After removing duplicate rows	1,349
After removing corrupted multivariate outlier	1,348

The final dataset contained no missing values, no duplicate observations, no infinite values, and valid binary labels only. It was therefore considered suitable for exploratory analysis and class impurity calculations.

2.2. Exploratory Data Analysis (EDA)

The EDA was conducted using summary statistics, histograms, boxplots, a class distribution chart, and a correlation heatmap. These visualizations help reveal the central tendency, variability, skewness, possible outliers, class balance, and relationships among the variables.

The cleaned dataset showed that the features had noticeably different distributions. X1 had a mean of approximately 0.446 and a median of approximately 0.519. Its standard deviation was 2.863, indicating moderate variability around the mean. The values of X1 ranged from approximately -7.04 to 6.82. The skewness of X1 was -0.158, which is close to zero. This suggests that the distribution of X1 is relatively balanced or approximately symmetric, although it is not perfectly normally distributed.

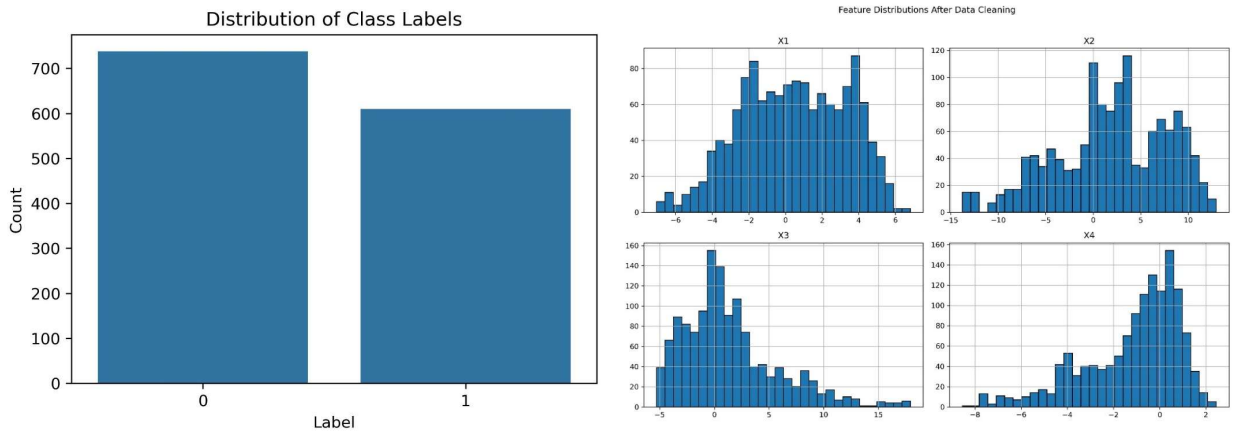

Figure 2: Distribution of class levels (left) and all four features X1, X2, X3, and X4 (right)

The feature X2 had the largest standard deviation among the predictors, approximately 5.869. Its values ranged from approximately -13.77 to 12.95. The histogram of X2 indicates that the variable has a broad distribution with observations spread across both negative and positive values. Its skewness was moderately negative, suggesting a somewhat longer lower tail.

The distribution of X3 was positively skewed, with a skewness value of approximately 1.088. Most observations of X3 were concentrated around lower to moderate values, while a smaller number of observations extended towards high positive values. The range of X3 was approximately -5.29 to 17.93. This positive skewness indicates that the right tail of X3 is longer than the left tail.

The distribution of X4 had a mean of approximately -1.169 and a median of approximately -0.579. Its standard deviation was 2.086, which was smaller than the standard deviation of X1. However, the distribution of X4 was clearly negatively skewed, with a skewness value of approximately -1.017. This indicates that the majority of X4 values were concentrated closer to zero or slightly below zero, while a relatively long tail extended towards highly negative values.

The class distribution plot showed that class 0 occurred slightly more frequently than class 1. There were 738 observations in class 0 and 610 observations in class 1. Although the dataset was not perfectly balanced, the class imbalance was not severe. Both classes were sufficiently represented for a binary classification problem.

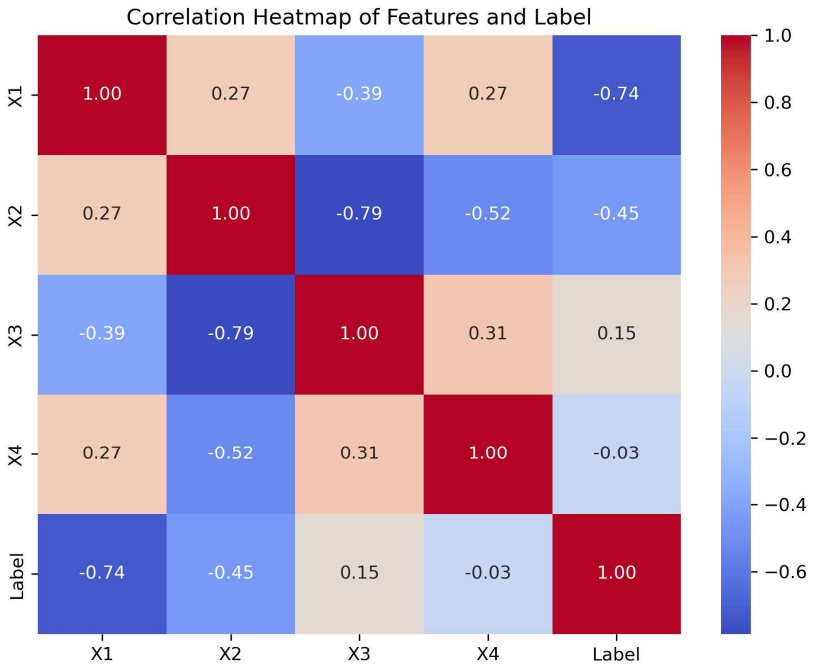

Figure 3: Correlation heatmap of all features

The correlation heatmap revealed several important linear relationships between the variables. The strongest negative correlation was between X2 and X3, with a correlation coefficient of approximately -0.79. This indicates that higher values of X2 tend to occur with lower values of X3, and vice versa. There was also a strong negative relationship between X1 and Label, with a correlation of approximately -0.74. This suggests that X1 may be a particularly informative predictor for distinguishing between the two classes. In contrast, X4 had almost no linear

correlation with Label, with a correlation of approximately -0.03. Therefore, X4 may not have a strong direct linear relationship with the target variable.

2.3. Key Difference Between Features X1 and X4

A major difference between the distributions of X1 and X4 is their shape and degree of skewness.

The histogram and boxplot of X1 show that its values are relatively evenly distributed around its central region. Its skewness value of -0.158 is close to zero, meaning that the distribution is approximately symmetric. X1 contains both negative and positive values, and its middle 50% of observations cover a relatively wide range. The interquartile range of X1 is approximately 4.64, which indicates substantial spread in the central part of the distribution.

In contrast, X4 is strongly negatively skewed. The histogram of X4 shows that a large proportion of values are concentrated around zero and slightly negative values, while the distribution extends further towards highly negative values. The boxplot of X4 also shows several low-value outliers below the lower whisker. Its skewness value of -1.017 confirms the visible left-tailed shape. The interquartile range of X4 is approximately 2.80, which is considerably smaller than the interquartile range of X1.

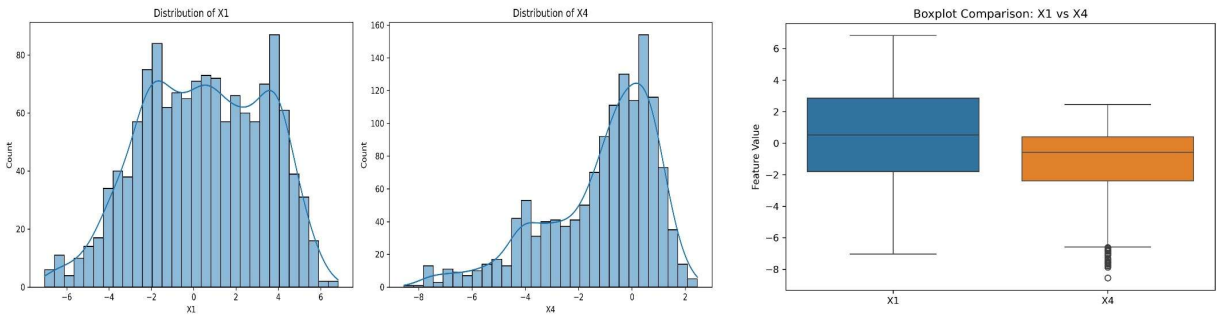

Figure 4: Comparison between feature X1 and X4. Data distribution in left side and box plot comparison in right side

Therefore, the key difference is that X1 is comparatively symmetric and has a wider central spread, whereas X4 is negatively skewed, more concentrated around its upper range, and contains a long lower tail with several low-value outliers. This difference is important because skewed variables may require transformations or robust modelling approaches in later predictive analysis.

2.4. Class Probabilities

The probabilities of the two classes were calculated using the number of observations in each class divided by the total number of observations in the cleaned dataset.

$$P(Label = 0) = \frac{738}{1348} = 0.5475$$

$$P(Label = 1) = \frac{610}{1348} = 0.4525$$

Thus, the probability of class 0 is approximately 54.75%, while the probability of class 1 is approximately 45.25%. This indicates that class 0 is the majority class, but the two classes remain relatively balanced.

2.5. Gini Impurity and Entropy

The Gini impurity (Breiman Leo, Jerome Friedman, Richard A. Olshen, and Charles J. Stone, 2017) was calculated using the following formula:

$$Gini = 1 - P(Label = 0)^2 - P(Label = 1)^2$$

$$Gini = 1 - (0.5475)^2 - (0.4525)^2$$

$$Gini = 0.4955$$

The Gini impurity is very close to its maximum value of 0.5 for a binary classification problem. This means that the classes are relatively mixed, and the dataset has high class uncertainty.

Entropy (Shannon, 1948) was calculated using the formula:

$$Entropy = -P(Label = 0)log_2\big(P(Label = 0)\big) - P(Label = 1)log_2(P(Label = 1))$$

$$Entropy = -[0.5475\ log_2(0.5475) + 0.4525\ log_2(0.4525)]$$

$$Entropy = 0.9935$$

The maximum entropy for a binary target is 1. Since the entropy value is 0.9935, the class distribution is very close to balanced. This confirms that neither class dominates the dataset strongly.

```
# Calculate class probabilities
class_counts = clean_df["Label"].value_counts().sort_index()

n_total = len(clean_df)

n_class_0 = class_counts[0]
n_class_1 = class_counts[1]

p_0 = n_class_0 / n_total
p_1 = n_class_1 / n_total

print("Total observations:", n_total)
print("Number of class 0:", n_class_0)
print("Number of class 1:", n_class_1)

print("\nProbability of class 0:", round(p_0, 4))
print("Probability of class 1:", round(p_1, 4))
```

```
# Calculate Gini Impurity
gini_impurity = 1 - (p_0 ** 2) - (p_1 ** 2)

print("Gini Impurity:", round(gini_impurity, 4))
```

```
Gini Impurity: 0.4955
```

```
# Calculate entropy
entropy = -(
    p_0 * np.log2(p_0) +
    p_1 * np.log2(p_1)
)

print("Entropy:", round(entropy, 4))
```

```
Entropy: 0.9935
```

```
# Predict the misclassification rate
majority_class = 0 if p_0 > p_1 else 1

misclassification_rate = 1 - max(p_0, p_1)

print("Majority class prediction:", majority_class)
print("Baseline Misclassification Rate:", round(misclassification_rate, 4))
print("Baseline Misclassification Rate (%):", round(misclassification_rate * 100, 2), "%")
```

Figure 5: Python code screenshot of calculating class probabilities, Gini Impurity, entropy, prediction of misclassification rate

2.6. Baseline Misclassification Rate

A simple baseline classifier would predict the majority class, class 0, for every observation. Under this approach, all 738 observations belonging to class 0 would be classified correctly. However, all 610 observations belonging to class 1 would be incorrectly classified as class 0.

Thus:

$$False\ Positives = 0$$

$$False\ Negatives = 610$$

$$Misclassification\ Rate = \frac{FP + FN}{N}$$

$$Misclassification\ Rate = \frac{0 + 610}{1348}$$

$$Misclassification\ Rate = 0.4525$$

Therefore, the baseline misclassification rate is approximately 45.25%. This means that a classifier that always predicts class 0 would make an incorrect prediction for almost 45 out of every 100 observations.

Chapter Three: Time Series Regression Analysis Using Polynomial Regression

3.1. Dataset Description

The dataset contains 1,000 rows and two columns:

Variable	Description	Data Type
t	Continuous time or input variable	Float
f(t)	Target variable to predict	Float

The dataset had the following characteristics:

- Number of observations: 1,000
- Number of variables: 2
- Missing values: None
- Duplicate rows: None
- Duplicate time values: None

Both variables were stored correctly as floating-point numerical values. Therefore, no imputation, encoding, or categorical transformation was required.

The descriptive statistics showed that the time variable ranged from approximately -4.98 to 4.99. The target variable ranged from approximately 0.00015 to 2092.28, with a mean of 276.43 and a relatively large standard deviation of 490.62. This large spread suggested a strong nonlinear relationship between *t* and *f(t)*.

3.2. Data Preprocessing

The preprocessing stage focused on ensuring that the data was suitable for time series regression.

First, the dataset was inspected to identify missing values, duplicate observations, duplicate time points, and incorrect data types. No missing values or duplicate rows were found. Since *t* represents

the ordered input variable, the data was sorted in ascending order based on *t*. Sorting was essential because the original dataset was not arranged chronologically.

```python
# Quality checks
print("Missing values:")
display(df[[TIME_COL, TARGET_COL]].isna().sum().to_frame("count"))

print("Duplicate rows:", df.duplicated().sum())
print("Duplicate time values:", df[TIME_COL].duplicated().sum())

# Remove only exact duplicates, if present, then sort chronologically.
df = df.drop_duplicates().copy()
df = df.dropna(subset=[TIME_COL, TARGET_COL]).copy()
df = df.sort_values(TIME_COL).reset_index(drop=True)

print("\nDescriptive statistics:")
display(df[[TIME_COL, TARGET_COL]].describe().T)

assert df[TIME_COL].is_monotonic_increasing, "Time must be sorted."
```

```python
# Distribution of target
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

sns.histplot(df[TARGET_COL], bins=35, kde=True, ax=axes[0])
axes[0].set_title("Distribution of f(t)")
axes[0].set_xlabel("f(t)")

sns.boxplot(x=df[TARGET_COL], ax=axes[1])
axes[1].set_title("Boxplot of f(t)")
axes[1].set_xlabel("f(t)")

plt.tight_layout()
plt.savefig("eda1.jpg", dpi=300, bbox_inches="tight")
plt.show()

# Observed relationship: original and logarithmic scale
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

axes[0].scatter(df[TIME_COL], df[TARGET_COL], s=16, alpha=0.75)
axes[0].set_title("Observed function over time")
axes[0].set_xlabel("t")
axes[0].set_ylabel("f(t)")
axes[0].grid(alpha=0.3)

axes[1].scatter(df[TIME_COL], df[TARGET_COL], s=16, alpha=0.75)
axes[1].set_yscale("log")
axes[1].set_title("Observed function over time (log scale)")
axes[1].set_xlabel("t")
axes[1].set_ylabel("f(t), log scale")
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig("eda2.jpg", dpi=300, bbox_inches="tight")
plt.show()
```

Figure 6: Jupyter notebook screenshot of Python code for data preprocessing (left) and exploratory data analysis (right)

No scaling or normalization was applied to the variables. Polynomial Regression can handle the original values directly, and preserving the original scale makes the results easier to interpret. However, feature transformation was performed through polynomial expansion, where powers of the input variable such as t^2, t^3, t^4, and t^5 were generated automatically.

The dataset was split chronologically into training and test sets. The first 80% of observations were used for training, while the final 20% of observations were reserved as a future hold-out test set. This approach is more appropriate than random splitting because it simulates a real forecasting scenario in which past observations are used to predict future observations.

The training dataset contained 800 observations with a time range from approximately -4.982 to 3.075. The test dataset contained 200 observations with a time range from approximately 3.078 to 4.990.

3.3. Exploratory Data Analysis

The exploratory data analysis revealed a highly nonlinear pattern between time and the target variable.

The observed function plot showed that the target value was very large for negative values of *t*. As time increased from approximately -5 toward zero, the function decreased sharply. The target then reached a value close to zero around $t \approx 0.268$.

After this first minimum point, the target increased gradually and reached a local maximum around $t = 2$, where the target value was approximately 9. After this local maximum, the function declined again and reached another value close to zero around $t \approx 3.732$. Finally, after this second minimum point, the target increased rapidly again as time approached 5.

The log-scale visualization was particularly useful because the target values varied greatly. On the original scale, the large values at negative time points made it difficult to observe the small values near the minima. The logarithmic plot clearly revealed the two low-value regions and the nonlinear curvature of the function.

The plots indicate that the data does not follow a linear trend. It also does not show a repeated seasonal cycle. Instead, the relationship follows a smooth curved pattern with two minima and one local maximum.

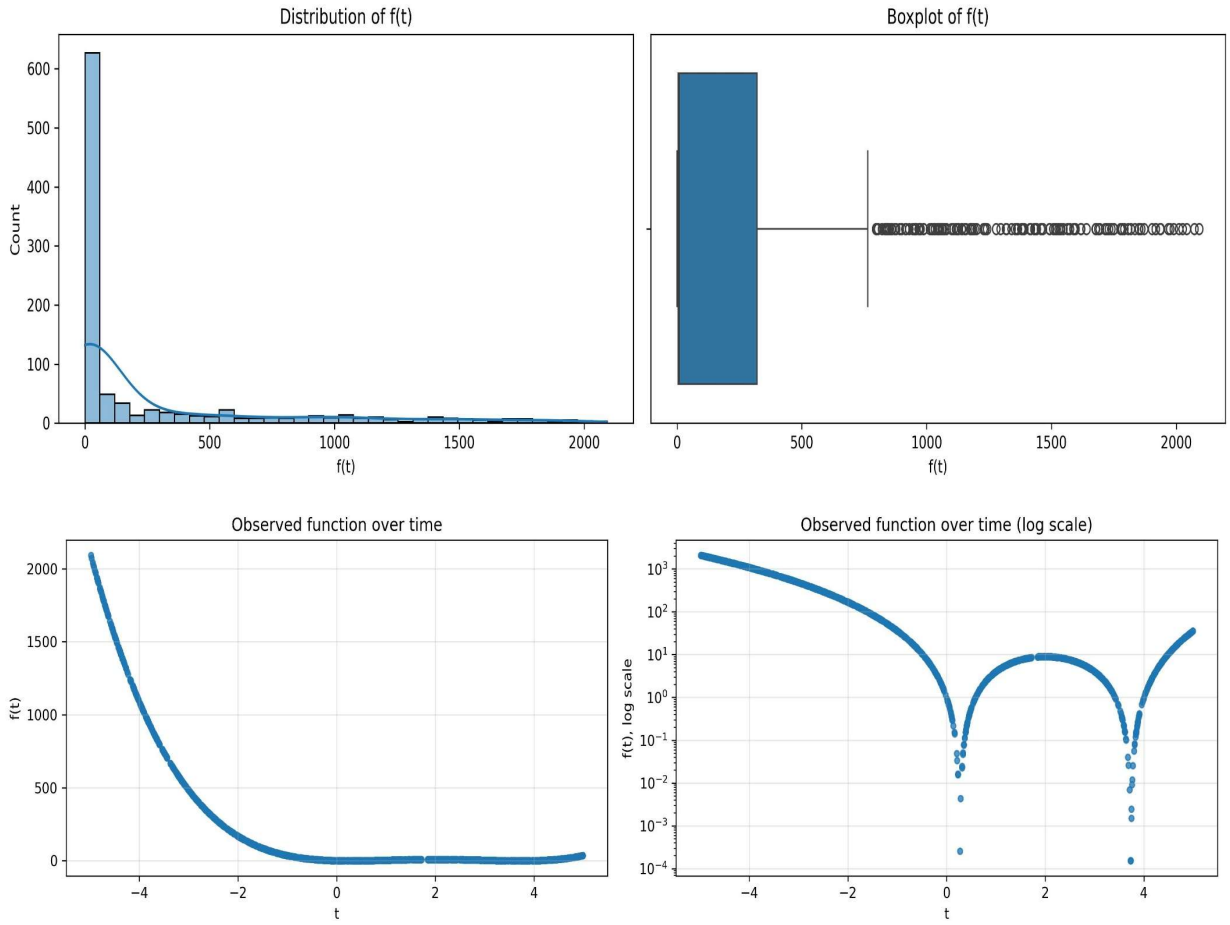

Figure 7: Data distribution of *f(t)* (top left), boxplot distribution of *f(t)* (top right); observed function trend over the time (bottom left), and log scale observed function trend over the time (bottom right)

3.4. Underlying Mathematical Trend

Further analysis showed that the data follows an almost exact polynomial relationship. The recovered polynomial coefficients were:

$$[1, -8{,}18, -8{,}1]$$

This corresponds to the following fourth-degree polynomial:

$$f(t) = t^4 - 8t^3 + 18t^2 - 8t + 1$$

The equation can also be expressed in a more interpretable form:

$$f(t) = (t^2 - 4t + 1)^2$$

This equation explains the shape observed in the plots. Because the function is squared, all output values remain non-negative. The two global minimum points occur when:

$$t^2 - 4t + 1 = 0$$

Solving this equation gives the two minimum locations:

$$t \approx 0.268$$

and

$$t \approx 3.732$$

At both locations, the value of the function is approximately zero.

The function also has a local maximum at:

$$t = 2$$

where:

$$f(2) = 9$$

Therefore, the observed time series follows a deterministic quartic function rather than a stochastic forecasting process.

3.5. Model Selection

Several polynomial degrees were evaluated using time-series cross-validation. Time-series cross-validation was used instead of random cross-validation because it preserves the temporal order of observations. In each fold, earlier observations were used for model training and later observations were used for validation.

The evaluated polynomial degrees (Draper, 1998) included linear, quadratic, cubic, fourth-degree, fifth-degree, sixth-degree, seventh-degree, and eighth-degree models.

The results showed that low-degree models performed poorly. For example, the linear regression model produced a high cross-validation RMSE of approximately 282.37, while the quadratic model produced an RMSE of approximately 61.98. This confirms that the relationship cannot be represented adequately using a simple linear or quadratic function.

The best cross-validation performance was achieved by polynomial degrees four and five. The fifth-degree model was selected because it had the lowest reported mean cross-validation RMSE, approximately 0.000004. The fourth-degree model also performed almost identically, which is expected because the true underlying relationship is a fourth-degree polynomial.

Although the fifth-degree model was selected through cross-validation, the discovered mathematical structure suggests that the fourth-degree polynomial is the most parsimonious representation of the actual function. The fifth-degree model provides extremely accurate predictions, but the fourth-degree equation offers a simpler interpretation of the data-generating process.

3.6. Model Evaluation

The selected fifth-degree Polynomial Regression model was trained using the 800 training observations and evaluated on the 200 future hold-out observations.

The test-set evaluation results were:

Metric	Result
Mean Absolute Error (MAE)	0.0000001440
Root Mean Squared Error (RMSE)	0.0000001829
R-squared ((R^2))	1.0000000000

The MAE and RMSE values are extremely close to zero, indicating that the predictions were almost identical to the actual values. The R^2 value of 1.0 indicates that the model explained virtually all variation in the target variable.

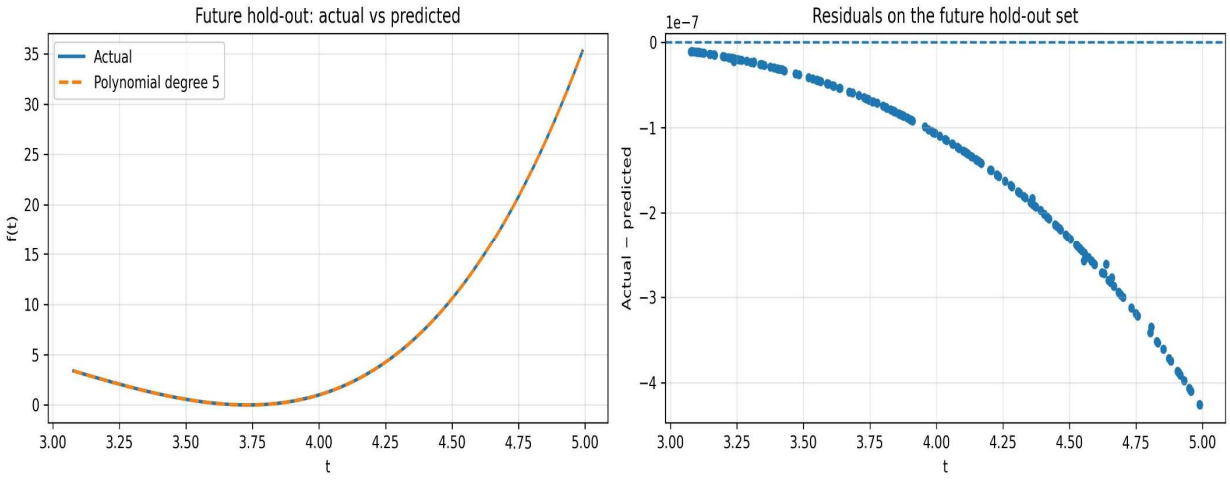

Figure 8: Actual value versus predicted values of Polynomial degree 5 model

The actual-versus-predicted plot confirms this result. The predicted curve almost completely overlaps with the actual curve throughout the future hold-out period. The model successfully captured the rapid increase in the target value after approximately t = 3.75.

The residual plot also supports the strong performance of the model. The residuals are extremely small, on the order of 10^{-7}. Although there is a slight curved residual pattern, its magnitude is practically negligible. This small pattern may occur because the selected model used degree five while the true underlying function is degree four.

Chapter Four: Conclusion

Data Quality Assessment, EDA, and Class Impurity Analysis

The classification dataset was cleaned by removing one observation with a missing target value, 25 duplicate observations, and one clearly corrupted multivariate outlier. After the cleaning process, the final dataset contained 1,348 valid observations suitable for further statistical analysis and machine learning modelling.

The exploratory data analysis showed that the four predictor variables had substantially different distributions in terms of shape, spread, skewness, and potential outliers. In particular, X1 was relatively symmetric and showed a wider central spread, whereas X4 was strongly negatively skewed, with a long lower tail and several low-value outliers. These differences indicate that feature-specific preprocessing and modelling considerations may be required in future classification tasks.

The class distribution was relatively balanced, with 54.75% of observations belonging to class 0 and 45.25% belonging to class 1. The Gini impurity value of 0.4955 and entropy value of 0.9935 both indicate high class uncertainty and a nearly balanced binary classification problem. Finally, a baseline classifier that always predicts the majority class would produce an estimated misclassification rate of 45.25%. This relatively high error rate demonstrates that a more advanced classification model using X1, X2, X3, and X4 is necessary to achieve useful predictive performance.

Time Series Regression Analysis Using Polynomial Regression

This task demonstrated the use of Polynomial Regression for modelling a nonlinear time series regression problem. The dataset was clean and required minimal preprocessing, apart from sorting observations according to time and applying a chronological train-test split to preserve the temporal ordering of the data.

The exploratory analysis showed a smooth nonlinear pattern with two global minima and one local maximum. The underlying relationship was found to be closely represented by the following deterministic function:

$$f(t) = (t^2 - 4t + 1)^2$$

Time-series cross-validation was used to compare Polynomial Regression models with different degrees. The fifth-degree Polynomial Regression model was selected because it achieved the lowest validation RMSE. When evaluated on the future hold-out test set, the model produced almost perfect predictions, with an (R^2) value of 1.0 and extremely low MAE and RMSE values.

The results confirm that Polynomial Regression is highly suitable for this dataset. Overall, this task highlights the importance of examining the underlying structure of a time-ordered dataset before selecting a modelling technique. Although the observations are arranged by time, the problem is best understood as a nonlinear functional regression task rather than a seasonal or autoregressive time series forecasting problem.

BIBLIOGRAPHY

Breiman Leo, Jerome Friedman, Richard A. Olshen, and Charles J. Stone, 2017. Classification and regression trees. *Chapman and Hall/CRC,.*

Draper, N., 1998. *Applied regression analysis.* s.l.:McGraw-Hill. Inc.

Shannon, C., 1948. A mathematical theory of communication. *The Bell system technical journal,* 27(3), pp. 379-423.